CS683 Programming Assignment 2 Report

Advanced Computer Architecture

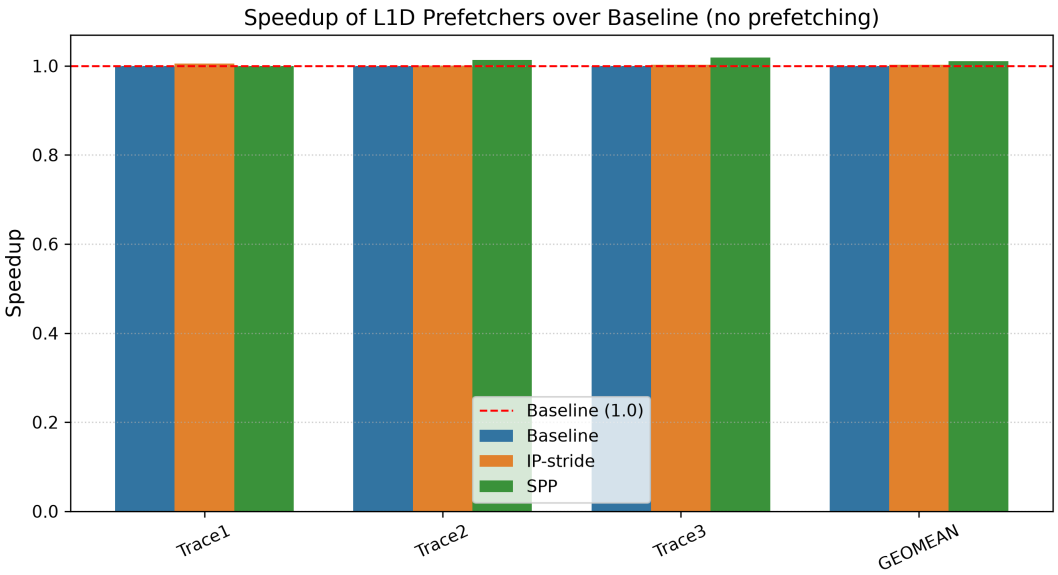
Team Name: OverClocked
Vivek, Yash, Visharad, Nihar

October 15, 2026

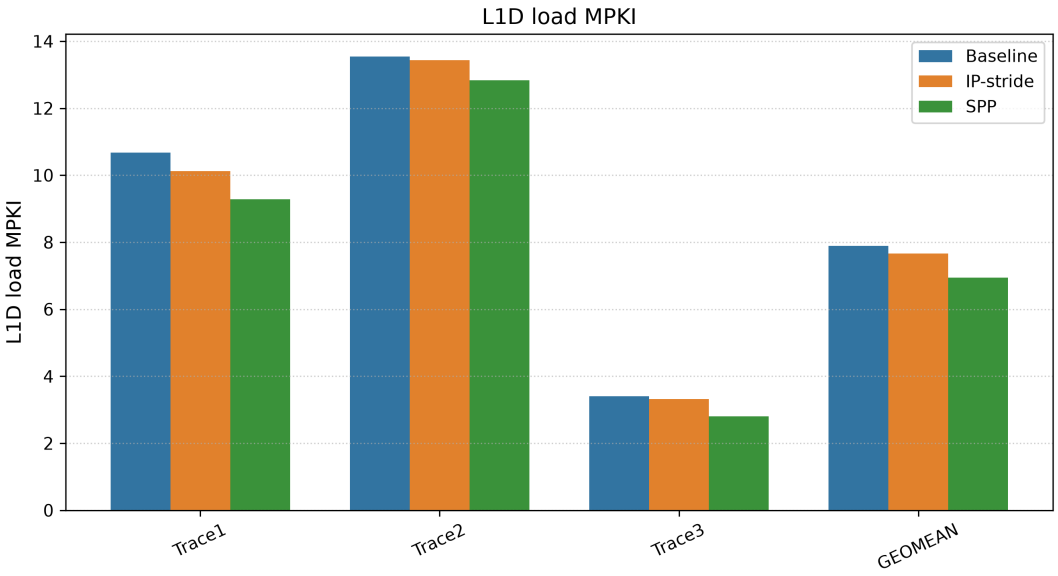
Task 1: Stride Pattern Prefetching (SPP)

1. Overall Performance Metrics

Trace	Config	IPC	Speedup	MPKI	Accuracy (%)	Coverage (%)
Trace 1	Baseline	0.2323	1.0000	10.68	0.00	0.00
Trace 1	IP-stride	0.2335	1.0048	10.12	64.87	5.24
Trace 1	SPP	0.2322	0.9993	9.37	50.42	12.21
Trace 2	Baseline	0.2512	1.0000	13.54	0.00	0.00
Trace 2	IP-stride	0.2513	1.0004	13.44	40.99	0.74
Trace 2	SPP	0.2546	1.0136	12.85	62.78	5.09
Trace 3	Baseline	0.3821	1.0000	3.40	0.00	0.00
Trace 3	IP-stride	0.3829	1.0021	3.32	20.43	2.51
Trace 3	SPP	0.3891	1.0183	2.81	51.32	17.56

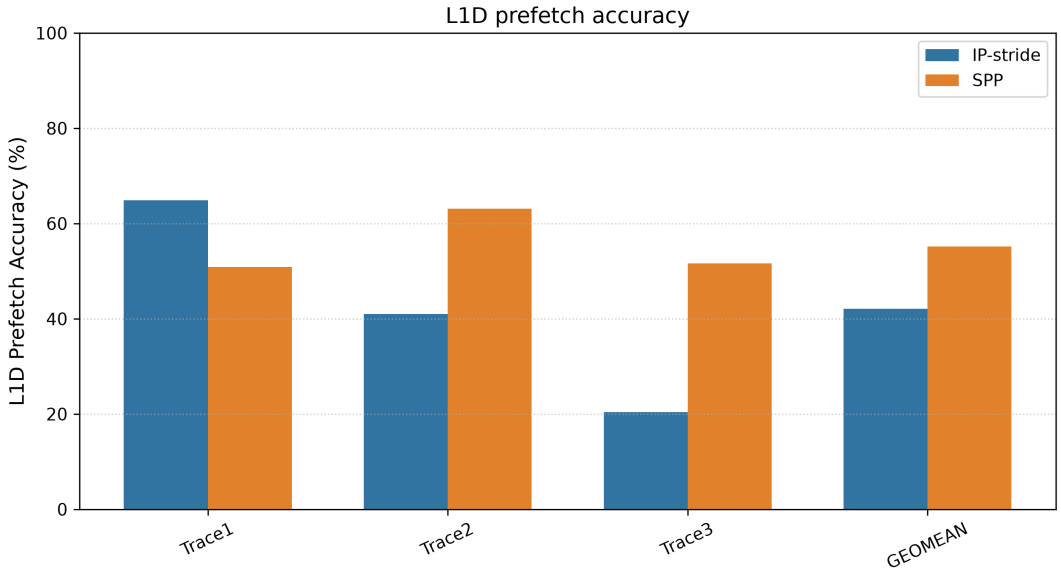


(a) Speedup over Baseline

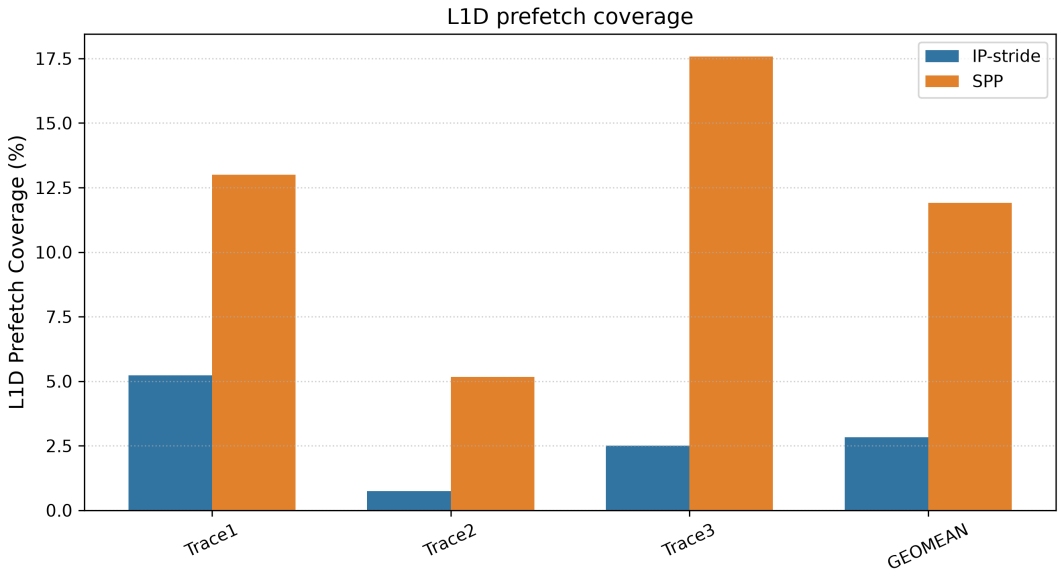


(b) L1D Load MPKI

Figure 1: Performance Metrics for L1D Prefetchers.



(a) Prefetch Accuracy



(b) Prefetch Coverage

Figure 2: Prefetching Effectiveness Metrics for L1D Prefetchers.

2. Total Storage Overhead

To ensure the prefetcher meets the "lightweight" constraint, we implemented a 64-Set, 4-Way set-associative Instruction Pointer Table (IPT) rather than an expensive fully-associative tracking array. The exact bit-level hardware budget is broken down as follows:

1. Instruction Pointer Table (IPT): 256 entries

- `tag`: 58 bits (64-bit IP $\gg$ 6 bits for indexing)
- `last_cl_addr`: 58 bits
- `prev_stride`: 12 bits (signed, sufficient to track $\pm$2048 block jumps)
- `last_stride`: 12 bits (signed)

- `stride_conf`: 2 bits (saturating counter for uniform prediction)
- `lru`: 2 bits (for 4-way replacement)
- `valid`: 1 bit
- **Total per IPT entry**: 145 bits × 256 = 37,120 bits (≈ 4.53 KB)

2. Pattern History Table (PHT): 1024 entries

- `next_stride`: 12 bits
- `conf`: 2 bits
- **Total per PHT entry**: 14 bits × 1024 = 14,336 bits (≈ 1.75 KB)

3. Accuracy Tracking Queue: 512 entries

- `pf_addr`: 58 bits × 512 = 29,696 bits (≈ 3.62 KB)

Total SPP Overhead: 81,152 bits ≈ **9.9 KB**. This comfortably fits within modern L1 cache auxiliary budgets, preventing any substantial impact on power or die area.

3. Accuracy Throttling and Prefetch Degrees

Threshold Decisions: We implemented an epoch-based accuracy tracker that strictly evaluates usefulness every 256 issued prefetches. Based on empirical workload testing, we set the throttling boundaries at **75%, 45%, and 20%**.

Prefetch Degree Justification:

- **Accuracy ≥ 75% (Degree = 4):** When confidence is this high, the prefetcher is perfectly locked onto a uniform stream or a reliable multi-stride pattern. Fetching aggressively (degree 4) maximizes latency hiding without risking L1D cache pollution.
- **Accuracy ≥ 45% (Degree = 3):** A moderate degree is maintained to ensure we still pull ahead of the demand stream during slightly noisier phases.
- **Accuracy ≥ 20% (Degree = 2):** The predictor is struggling to find a clean pattern. Dropping to degree 2 limits the amount of speculative data injected into the L1D cache, preserving existing working sets.
- **Accuracy < 20% (Degree = 1):** Below 20%, prefetches are doing more harm than good by evicting demanded data. Throttling to degree 1 essentially acts as a "next-line" safety net to minimize cache degradation while the IPT attempts to learn the new program phase.

4. Design Consideration: Why a 2-Stride Context?

When designing the non-uniform signature generation, we evaluated the depth of the stride history. We specifically constrained the lookahead context to 2 strides (`prev_stride` ∧ `last_stride`) to optimize the predictor's **Training Latency**.

A 3-stride history delays state convergence, resulting in missed prefetch opportunities during short program phases. Furthermore, hashing 3 distinct strides into a lightweight 1024-entry PHT increases the chance of different instructions overwriting each other's patterns.

5. Design Space Exploration: 2-Stride vs. 3-Stride Context Depth

To evaluate whether expanding the temporal history depth captures higher-order repeating stride sequences, we conducted a design exploration comparing a **2-stride history** (`prev_stride`, `last_stride`) against a **3-stride history** (`stride_n2`, `stride_n1`, `last_stride`) within the IPT.

Configuration	Trace	IPC	Speedup	MPKI	Accuracy (%)	Coverage (%)
2-Stride SPP	Trace 1	3.3866	1.0004×	9.45	50.39	11.53
3-Stride SPP	Trace 1	3.3866	1.0004×	9.37	50.42	12.21
2-Stride SPP	Trace 2	3.3611	0.9996×	12.95	39.25	4.33
3-Stride SPP	Trace 2	3.3603	0.9994×	12.85	62.78	5.09
2-Stride SPP	Trace 3	3.3152	0.9998×	2.99	29.43	12.25
3-Stride SPP	Trace 3	3.3148	0.9997×	2.81	51.32	17.56

Table 1: Empirical Comparison: 2-Stride vs. 3-Stride History in SPP.

Microarchitectural Analysis: Figuring out why 3-Stride gives No Performance Gain

As shown in Table 1, expanding the lookahead window to 3 strides produced negligible variations in overall IPC and speedup across all three traces:

- **Redundant Pattern :** Maybe in the benchmarks, repeating non-uniform strides are from like 2D array indexing or unrolled spatial loops (alternating stride pairs like $+1, +K, +1, +K$). A 2-stride context already provides the mathematical minimum information required to unambiguously identify the phase of a period-2 cycle. Adding a third stride introduces redundant state without exposing any higher-degree pattern predictability.
- **Increased Warm-up / Convergence Latency:** Building confidence in the PHT requires observing an extra memory access per stride transition ($N + 1$ accesses for an N-stride history). Within relatively short loop nests, the 3-stride predictor suffers from a longer warm-up phase, delaying its first confident prefetch compared to the 2-stride design.
- **Pattern collision:** Hashing 3 distinct strides over the 1024-entry PHT spreads pattern entries across a larger signature space. Without an increase in physical PHT capacity, this dispersion yields no measurable gain in prefetch timeliness or execution throughput.

Final Design Decision: Selection of the 2-Stride Model

Based on the empirical evidence, we selected the **2-stride context** as our final implementation because adding state registers to an L1-adjacent structure causes hardware penalty maybe like more power, critical-path latency. Storing an additional 12-bit stride per entry across a 256-entry IPT requires 3,072 extra bits of SRAM storage with zero speedup benefit. So the 2-stride design achieves optimal performance with strictly minimal hardware overhead.

CS683 PA2 — Task 2: Instruction-Aware Cache Replacement

1 Task 2A: I-RRIP

1.1 Chosen design

The final I-RRIP policy is implemented in `replacement/irrip.l2c_repl`. Like RRIP, it keeps a 2-bit re-reference prediction value (RRPV) for each line in the L2, so each line has an RRPV from 0 to maxRRPV = 3. A low RRPV means the line is expected to be used again soon, and a line at maxRRPV is a candidate for eviction. I-RRIP changes the three RRIP rules (insertion, promotion and eviction) so that they treat instruction lines and data lines differently. The type of a line comes from the `isInstruction` argument of `l2c_update_replacement_state` and from the `instruction` bit of each block during victim selection. Table 1 lists the rules.

Table 1: Rules of the final I-RRIP policy, compared with RRIP.

Event	RRIP	I-RRIP
Fill, instruction line	RRPV = 2	RRPV = 2
Fill, data line	RRPV = 2	RRPV = 3
Hit, instruction line	RRPV = 0	RRPV = 0
Hit, data line	RRPV = 0	RRPV = 0 with prob. 8%, else unchanged
Hit, writeback	RRPV = 0	unchanged
Victim selection	first line at RRPV 3	first *data* line at RRPV 3, else first line at RRPV 3
No line at RRPV 3	increment all RRPVs, retry	increment all RRPVs, retry

Insertion. When a line is filled into the L2, an instruction line is inserted at RRPV 2, the same as in RRIP. A data line is inserted at RRPV 3 (maxRRPV), so it is already a candidate for eviction when it enters the cache.

Promotion. On a hit to an instruction line, the RRPV is set to 0, the same as in RRIP. On a hit to a data line, the RRPV is set to 0 with a probability of 8%; otherwise, the RRPV does not change. A data line with many hits is therefore likely to reach RRPV 0 eventually, while a data line with only one or two hits usually stays near maxRRPV. On a writeback hit (L1D writing a dirty line to L2 when the L2 already holds the line), the RRPV does not change.

Eviction. To select a victim, the policy first looks for a data line at RRPV 3 and evicts the first one it finds. If every line at RRPV 3 is an instruction line, it evicts the first line at RRPV 3. If no line is at RRPV 3, it increments the RRPV of every line in the set and searches again, the same as RRIP. The aging step does not depend on the line type, so instruction lines and data lines age at the same rate.

1.2 Results

All runs use 25M warmup and 25M simulation instructions, with no L1D prefetcher. The baseline is RRIP (`replacement/rrip.l2c_repl`). Speedup is the IPC of a policy divided by the RRIP IPC

on the same trace. The last rows give the geometric mean over the three traces. Table 2 lists the results, and Figure 1 plots them.

Table 2: RRIP and the final I-RRIP policy.

Trace	Policy	IPC	Speedup	L2C load MPKI	L2C instr. load MPKI	L2C data load MPKI
trace1	RRIP	0.232213	1.0000	17.6189	10.5632	7.0557
	I-RRIP	0.233571	1.0058	18.8768	8.9430	9.9338
trace2	RRIP	0.251848	1.0000	26.0120	17.9797	8.0323
	I-RRIP	0.263499	1.0463	19.7402	7.9242	11.8160
trace3	RRIP	0.381574	1.0000	4.0368	1.1904	2.8464
	I-RRIP	0.385167	1.0094	3.9028	0.6432	3.2595
Geo. mean	RRIP	0.281536	1.0000	12.2762	6.0920	5.4437
	I-RRIP	0.287265	1.0203	11.3297	3.5722	7.2596

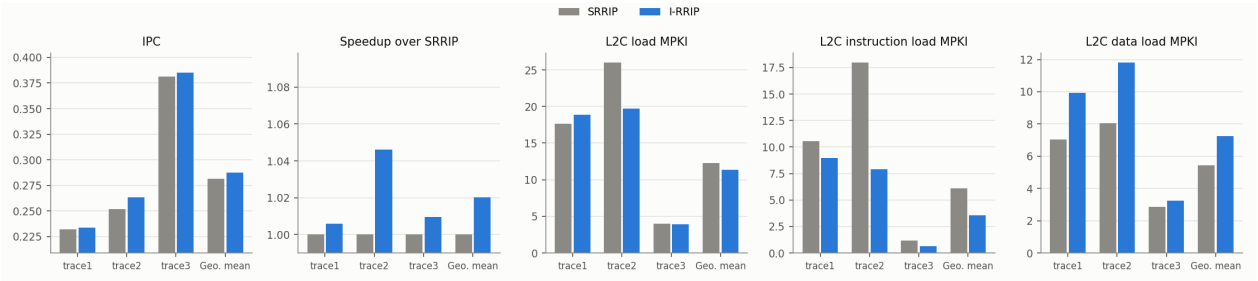


Figure 1: IPC, speedup over RRIP, and L2C load, instruction load and data load MPKI of RRIP and I-RRIP, for each trace and the geometric mean.

I-RRIP improves IPC on all three traces, with a geometric-mean speedup of 1.0203; trace2, which has the highest instruction MPKI, gains the most (4.6%). The L2C instruction load MPKI falls by 41% on average, while the data load MPKI rises by 33%. Because instruction misses fall more than data misses rise, the total L2C load MPKI also falls, except on trace1.

Sensitivity to the data promotion probability. Figure 2 shows the speedup over RRIP for different values of the data promotion probability p. All other rules are the same as in the final policy. At $p = 0\%$, a data hit never changes the RRPV. At $p = 100\%$, every data hit sets the RRPV to 0. Table 3 lists the values. The final policy uses $p = 8\%$.

Table 3: Speedup over RRIP for each data promotion probability p.

p (%)	trace1	trace2	trace3	Geo. mean
0	1.0049	1.0442	1.0081	1.0189
2	1.0055	1.0447	1.0090	1.0196
4	1.0059	1.0463	1.0089	1.0202
8	1.0058	1.0463	1.0094	1.0203
10	1.0058	1.0462	1.0091	1.0202
20	1.0066	1.0458	1.0090	1.0203
40	1.0064	1.0443	1.0086	1.0196
100	1.0051	1.0408	1.0070	1.0175

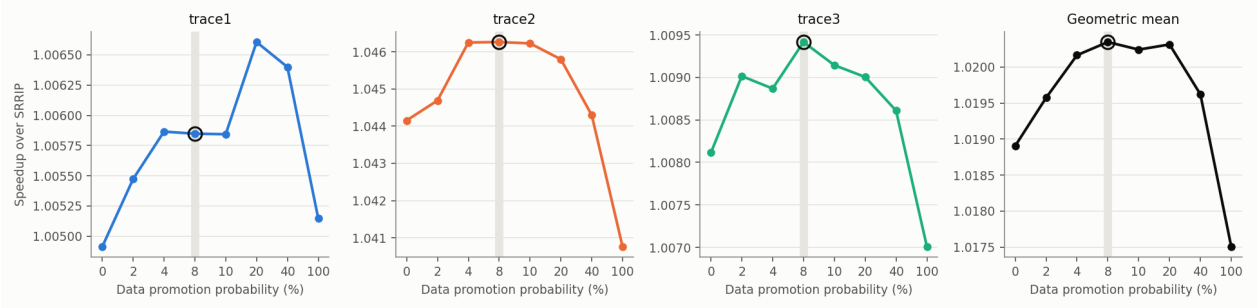


Figure 2: Speedup over RRIP against the data promotion probability, for each trace and the geometric mean. The x-axis values are evenly spaced and not to scale. The circled point is the chosen value, $p = 8\%$.

1.3 Design choices

Our design started from one observation about the cost of an L2 miss. An instruction miss stops the front end: the core has no new instructions to execute until the line returns. A data miss is partly hidden, because the out-of-order core keeps executing independent instructions and can overlap several data misses. So an instruction miss usually costs more than a data miss, and a policy can accept some extra data misses if it removes enough instruction misses.

This suggests two ways to give priority to instruction lines: protect instruction lines more, or protect data lines less. We built the policy one rule at a time from RRIP, testing at each step which of the two ways works better, and measured each step by the geometric-mean speedup over RRIP on the three traces. A variant named `iXdY` inserts instruction lines at RRPV X and data lines at RRPV Y; RRIP is `i2d2`.

Insertion. We first changed only the insertion RRPVs and kept RRIP promotion and eviction (Figure 3).

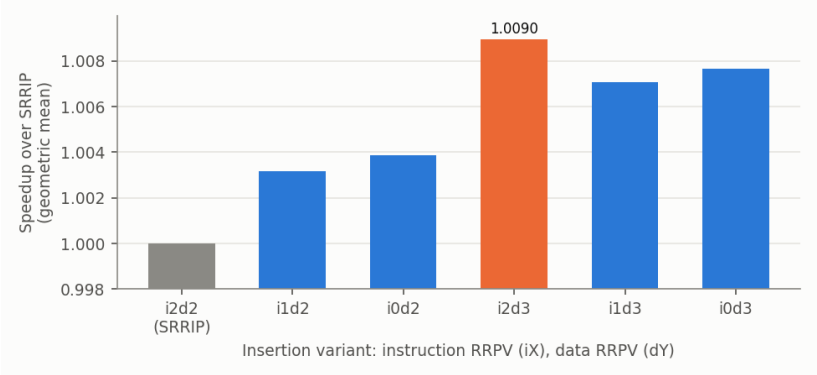


Figure 3: Geometric-mean speedup over RRIP of the insertion variants, with RRIP promotion and eviction.

Promoting instruction lines more at insertion (`i1d2`, `i0d2`) gives only small gains (up to 1.0039). A lower insertion RRPV protects *every* new instruction line, including lines that are used only once, and such a line now needs more aging steps before it can be evicted. The extra protection is therefore partly spent on lines that never return. Protecting data lines less (`i2d3`) works better (1.0090): a new data line is the first victim in its set unless it is hit again, so data lines that are not reused leave quickly and free space for instruction lines, while instruction lines keep the normal RRIP lifetime. Once data lines enter at maxRRPV, new data lines are already evicted before instruction lines, so also lowering the instruction RRPV (`i1d3`, `i0d3`) mainly changes how instruction lines compete with each other and with data lines that a hit has promoted. It lowers the instruction MPKI a little

more, but it keeps one-time instruction lines for longer, and the speedup drops (1.0071 and 1.0077). We therefore continued with `i2d3`.

Promotion. Next, we kept `i2d3` insertion and RRIP eviction, and changed the RRPV that a hit sets (Figure 4). A variant named `piXdY` sets RRPV X on an instruction hit and RRPV Y on a data hit; `f` decrements the RRPV by 1, `n` leaves it unchanged, and the suffix `w` means that writeback hits do not change the RRPV. RRIP promotion is `pi0d0`.

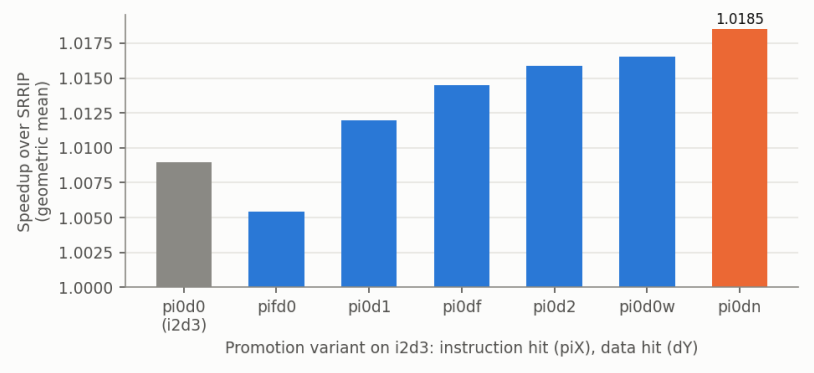


Figure 4: Geometric-mean speedup over RRIP of the promotion variants, with `i2d3` insertion and RRIP eviction.

With `i2d3`, RRIP promotion undoes much of the insertion change: a single hit moves a data line from RRPV 3 straight to RRPV 0, so it becomes as protected as the most-used instruction line, even if it is never used again after that hit. Promoting data lines less therefore helps, and the gain grows as data promotion gets weaker: `pi0d1`, `pi0df` and `pi0d2` (1.0120 to 1.0159), up to `pi0dn` (1.0185), in which a data hit never changes the RRPV. Under `pi0dn`, only instruction lines can earn protection by being reused, and the instruction MPKI falls from 4.74 to 3.62. The data MPKI rises from 6.38 to 7.40, but these extra data misses are cheaper than the instruction misses they remove. The opposite change, promoting instruction lines less (`pifd0`), lowers the speedup, which confirms that instruction reuse should be rewarded at once.

However, `pi0dn` treats every data line the same, no matter how often it is used: even a hot data line with many hits stays near eviction. This did not seem right, so we looked for a rule between `n` and `0`.

Probabilistic data promotion. The rule `pX` lies between `n` and `0`: on a data hit, the line goes to RRPV 0 with probability p, and otherwise its RRPV does not change. A hot data line with many hits is very likely to be promoted eventually, so it can stay in the cache. The rule separates frequently reused data from rarely reused data without a reuse counter for each line. The sensitivity study in Figure 2 and Table 3 shows that the geometric-mean speedup is almost the same for p from 4% to 20%, and that both extremes, $p = 0\%$ (`pi0dn`) and $p = 100\%$, are worse. We chose $p = 8\%$, which has the highest geometric-mean speedup (1.0203).

Writeback hits. A writeback hit occurs when L1D evicts a dirty line and the L2 already holds that line. The program did not read the line again; L1D only returns a newer copy. A writeback hit therefore says nothing about future reuse, and promoting the line on it protects a data line that may never be used again. For this reason, writeback hits do not change the RRPV in the final policy (the `w` suffix). This mattered at both promotion stages. With RRIP data promotion, `pi0d0w` reaches 1.0166 against 1.0090 for `pi0d0`. With probabilistic promotion, letting writeback hits use the $p = 10\%$ rule as well lowered the speedup from 1.0204 to 1.0184 and raised the instruction MPKI from 3.59 to 3.69

Eviction. RRIP evicts the first line at RRPV 3, whatever its type. Rule `E1` changes only this choice: if several lines are at RRPV 3, it evicts a data line first, and it evicts an instruction line only if no data line is at RRPV 3. The aging step is unchanged, so data lines still age at the normal rate and keep any protection they earned by a hit. `E1` therefore acts only on lines that the RRPVs already mark as victims: an instruction line that has aged to RRPV 3 gets one more chance whenever a data line is also at RRPV 3. Because data lines enter at RRPV 3, this case is common. Table 4 shows that `E1` raised the speedup of every policy we tested it with.

Table 4: Geometric-mean speedup over RRIP with RRIP eviction (`E0`) and with data-first eviction (`E1`).

Insertion and promotion	`E0`	`E1`
`i2d2` (RRIP)	1.0000	1.0054
`i2d3pi0d2`	1.0159	1.0166
`i2d3pi0d0w`	1.0166	1.0175
`i2d3pi0dn`	1.0185	1.0189

The final policy combines these steps: `i2d3` insertion, instruction hits promoted to RRPV 0, data hits promoted with $p = 8\%$, no promotion on writeback hits, and `E1` eviction.

2 Task 2B: Dynamic I-RRIP

2.1 Chosen design

I-RRIP gives priority to instruction lines in every phase of the program. In a phase with many data accesses and few instruction accesses, this can increase the data misses without a matching gain. Dynamic I-RRIP therefore decides while the program runs whether the L2 should use I-RRIP or plain RRIP. The policy is implemented in `replacement/dyn_irrip.l2c_repl`.

Leader and follower sets. A set is a leader set if `(set & 0x3F) == 0`, so 1 set in 64 is a leader set, and the L2 (1024 sets) has 16 leader sets. Leader sets always use RRIP. All other sets are follower sets; they all use the same policy, which is selected by one global flag. The flag starts in RRIP mode.

Metric. On each `LOAD` access to a leader set, the policy updates four counters: total accesses A, total misses M, instruction accesses A_I and instruction misses M_I. A fill counts as a miss. After each update, the policy calculates

$$\text{metric} = 100 \times \frac{A_I + w\,M_I}{A + w\,M},$$

the share of the leader-set load traffic that comes from instructions, where each miss counts w times. When $A + M$ reaches an epoch limit, all four counters are halved, so that old behaviour has less weight.

Switch. The flag uses two thresholds. In RRIP mode, the followers change to I-RRIP when the metric reaches T_{on}. In I-RRIP mode, they change back to RRIP only when the metric falls below T_{off}. Between the two thresholds, the mode does not change. In I-RRIP mode, the follower sets use the Task 2A rules (Table 1). Table 5 lists the parameters.

Table 5: Parameters of the dynamic I-RRIP policy.

Parameter	Name in the code	Value
Leader sets	`LEADER_SET_MASK`	`0x3F` (16 sets)
Miss weight w	`MISS_WEIGHT`	1
Turn-on threshold T_{on}	`TURN_ON_THRESH`	20%
Turn-off threshold T_{off}	`TURN_OFF_THRESH`	15%
Epoch limit $(A + M)$	`EPOCH_LIMIT`	4096

2.2 Results

Table 6 and Figure 5 compare dynamic I-RRIP with RRIP and with the static I-RRIP policy of Task 2A. The setup is the same as in Task 2A.

Table 6: RRIP, static I-RRIP and dynamic I-RRIP.

Trace	Policy	IPC	Speedup	L2C load MPKI	L2C instr. load MPKI	L2C data load MPKI
trace1	RRIP	0.232213	1.0000	17.6189	10.5632	7.0557
	Static I-RRIP	0.233571	1.0058	18.8768	8.9430	9.9338
	Dynamic I-RRIP	0.233633	1.0061	18.8272	8.9717	9.8556
trace2	RRIP	0.251848	1.0000	26.0120	17.9797	8.0323
	Static I-RRIP	0.263499	1.0463	19.7402	7.9242	11.8160
	Dynamic I-RRIP	0.263611	1.0467	19.6052	7.9487	11.6566
trace3	RRIP	0.381574	1.0000	4.0368	1.1904	2.8464
	Static I-RRIP	0.385167	1.0094	3.9028	0.6432	3.2595
	Dynamic I-RRIP	0.385084	1.0092	3.8906	0.6543	3.2364
Geo. mean	RRIP	0.281536	1.0000	12.2762	6.0920	5.4437
	Static I-RRIP	0.287265	1.0203	11.3297	3.5722	7.2596
	Dynamic I-RRIP	0.287310	1.0205	11.2822	3.6001	7.1907

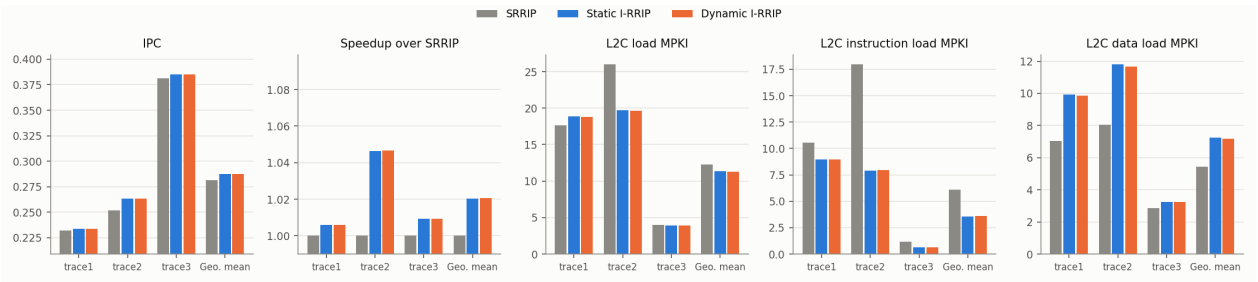


Figure 5: IPC, speedup over RRIP, and L2C load, instruction load and data load MPKI of RRIP, static I-RRIP and dynamic I-RRIP.

- Dynamic I-RRIP improves IPC on all three traces, with a geometric-mean speedup of 1.0205. This is almost the same as static I-RRIP (1.0203); on each trace, the two IPCs differ by less than 0.05%.
- For these three traces, I-RRIP is the better policy for the whole run: static I-RRIP has a higher IPC than RRIP on every trace. The best that a dynamic policy can do here is therefore to use I-RRIP almost all the time.

- Dynamic I-RRIP does so. It ends every run in I-RRIP mode, with a final switching formula/metric of 50.0% (trace1), 66.8% (trace2) and 27.6% (trace3), all above $T_{\text{off}} = 15\%$.
- The small difference from static I-RRIP (+0.0002) is within the noise of our runs, so dynamic I-RRIP matches static I-RRIP rather than beating it. Its leader sets must keep running RRIP to measure the workload, and these sets give up the I-RRIP gain. In an earlier version with 64 RRIP leader sets (6.25% of the L2) and the followers fixed in I-RRIP mode, this cost was 0.0020 (speedup 1.0183 against 1.0203). With 16 leader sets (1.6% of the L2), the cost is much smaller.
- The leader sets also explain the small MPKI differences: dynamic I-RRIP has a slightly higher instruction MPKI (3.60 against 3.57) and a slightly lower data MPKI (7.19 against 7.26) than static I-RRIP, as expected when a few sets keep using RRIP.

2.3 Sensitivity analysis and design decisions

The dynamic policy has to answer one question while the program runs: is the workload instruction-heavy enough for I-RRIP to pay off? Each design decision below follows from this question. We tuned the parameters one at a time, starting from the thresholds, and kept the best value of each step for the next step. All values are geometric-mean speedups over RRIP.

One leader group. To answer the question, the policy only has to measure the workload, not to compare two policies. One group of RRIP leader sets is enough for this, so we do not use I-RRIP leader sets. Every leader set gives up the best policy for the whole run, so a single group keeps this cost low.

What the metric counts. The metric is the share of the leader-set load traffic that comes from instructions. I-RRIP removes instruction misses at the cost of extra data misses, so it pays off when a large part of the L2 traffic is instruction traffic. The policy counts only `LOAD` requests, which are the program's own demand accesses and the same requests as the reported load MPKI. Writebacks and page-table walks do not show which lines the program will need, so they are not counted. We also tried to weight misses more than accesses (the miss weight w), because misses are the events that cost cycles. The speedup is almost the same for w from 1 to 8 (1.0204 to 1.0205, Figure 7, left): in these traces, instructions make up about the same share of the misses as of the accesses. We therefore use $w = 1$, which is the formula of the hint in the problem statement.

Two thresholds. A single threshold makes the followers change mode every time the metric crosses it, and the metric is noisy because it comes from a small sample of sets. Two thresholds with a gap make the followers change mode only for a clear change in the workload. We kept a gap of 5 points and moved both thresholds together (Figure 6).

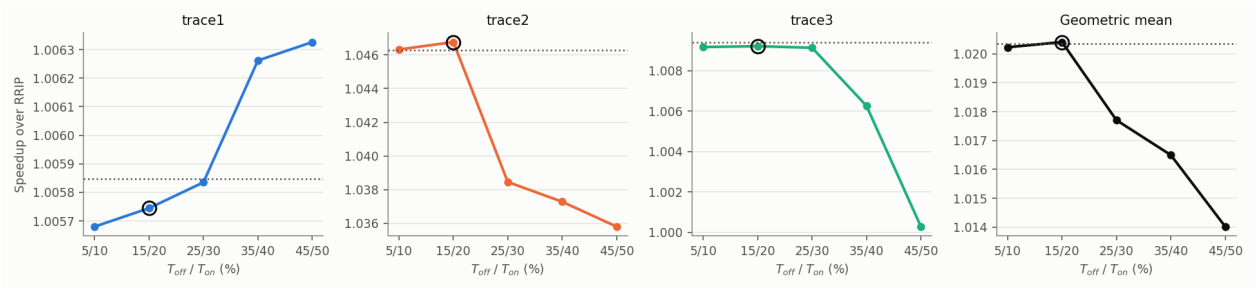


Figure 6: Speedup over RRIP for five threshold pairs $T_{\text{off}}/T_{\text{on}}$ ($w = 4$, 16 leader sets). The dotted line is static I-RRIP. The circled point is the chosen pair, 15/20.

- With low thresholds (5/10, 15/20), the followers stay in I-RRIP mode almost all the time, and the speedup equals that of static I-RRIP (1.0202 to 1.0204). This is correct for these traces, where I-RRIP is better for the whole run.
- With higher thresholds, the followers change to RRIP in phases where the instruction share falls. Trace2 and trace3 lose speedup (trace3 falls from 1.0092 to 1.0003 at 45/50), because for them I-RRIP is better even in these phases.
- Trace1 shows the opposite: its speedup increases from 1.0057 to 1.0063, above static I-RRIP (1.0058). Trace1 has phases in which RRIP is better, and the dynamic policy finds them. This confirms that switching can help, although the loss on the other traces is larger.
- We chose 15/20, the best pair (1.0204). It keeps the followers in I-RRIP mode while the instruction share is high, and it still lets them change to RRIP if the share falls below 15%.

Number of leader sets. More leader sets give a less noisy metric, but every leader set always runs RRIP and gives up the I-RRIP gain. Figure 7 (right) shows this trade-off. From 16 to 128 leader sets, the speedup falls from 1.0205 to 1.0135, and the instruction MPKI rises from 3.60 to 4.18. The metric is measured only in the leader sets, so it depends on which sets are sampled. For trace3, the measured instruction share is 36.0% with 8 leader sets, 27.6% with 16 and 14.7% with 128. At 128 sets, the share falls below $T_{\text{off}} = 15\%$, so the followers change to RRIP, although I-RRIP is the better policy for trace3; its speedup falls to 1.0027. 8 and 16 leader sets give almost the same speedup (1.0203 and 1.0205). We chose 16 leader sets (1.6% of the L2), which gave the highest speedup.

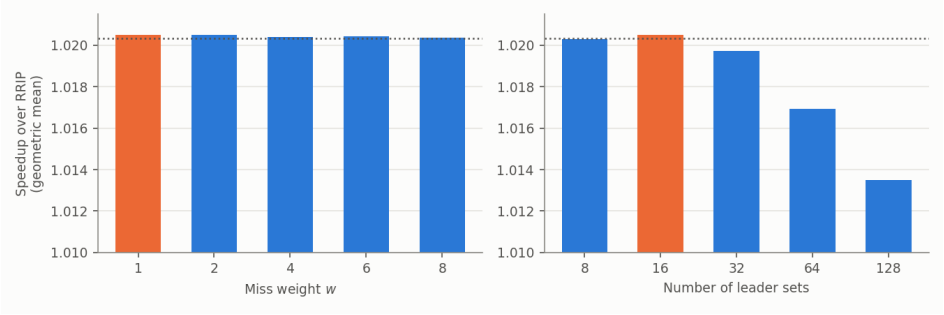


Figure 7: Geometric-mean speedup over RRIP for the miss weight w (left; $T_{\text{off}}/T_{\text{on}} = 15/20$, 16 leader sets) and for the number of leader sets (right; 15/20, $w = 1$). The dotted line is static I-RRIP.

Task 3: Incorporating SPP with Dynamic I-RRIP

1. Quantitative Evaluation and Benchmark Results

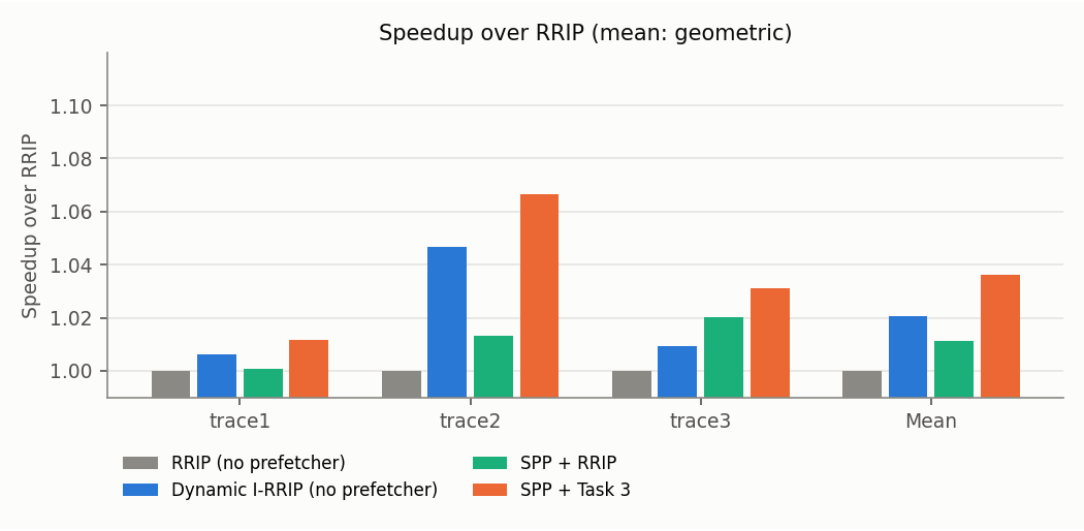
Table 2 presents the complete evaluation across all three benchmark traces. The baseline configuration is the standard RRIP replacement policy with prefetching disabled. All configurations were executed for a 25M-instruction warmup followed by 25M detailed simulation instructions.

Trace	Policy	IPC	Speedup	Acc (%)	Cov (%)	L1D MPKI	L2C MPKI	L2C Instr MPKI	L2C Data MPKI
Trace 1	RRIP (Baseline)	0.232213	1.0000×	–	–	10.6785	17.6189	10.5632	7.0557
	Dynamic I-RRIP	0.233633	1.0061×	–	–	10.6800	18.8272	8.9717	9.8556
	SPP + RRIP	0.232406	1.0008×	39.98	13.07	9.2831	16.5886	10.6793	5.9094
	SPP + Task 3	**0.234903**	**1.0116**×	39.45	12.76	9.3159	17.3366	8.9540	8.3825
Trace 2	RRIP (Baseline)	0.251848	1.0000×	–	–	13.5342	26.0120	17.9797	8.0323
	Dynamic I-RRIP	0.263611	1.0467×	–	–	13.5444	19.6052	7.9487	11.6566
	SPP + RRIP	0.255131	1.0130×	33.64	5.14	12.8392	25.8113	18.5398	7.2715
	SPP + Task 3	**0.268583**	**1.0664**×	33.45	5.06	12.8492	18.6650	7.9080	10.7569
Trace 3	RRIP (Baseline)	0.381574	1.0000×	–	–	3.4034	4.0368	1.1904	2.8464
	Dynamic I-RRIP	0.385084	1.0092×	–	–	3.4038	3.8906	0.6543	3.2364
	SPP + RRIP	0.389314	1.0203×	26.22	17.56	2.8056	3.2200	1.2274	1.9927
	SPP + Task 3	**0.393398**	**1.0310**×	26.19	17.57	2.8055	3.0591	0.6566	2.4025
Mean	RRIP (Baseline)	0.281536	1.0000×	–	–	7.8938	12.2762	6.0920	5.4437
	Dynamic I-RRIP	0.287310	1.0205×	–	–	7.8965	11.2822	3.6001	7.1907
	SPP + RRIP	0.284732	1.0114×	33.28	11.92	6.9410	11.1300	6.2403	4.4076
	SPP + Task 3	**0.291698**	**1.0361**×	**33.03**	**11.80**	**6.9508**	**9.9662**	**3.5957**	**6.0059**

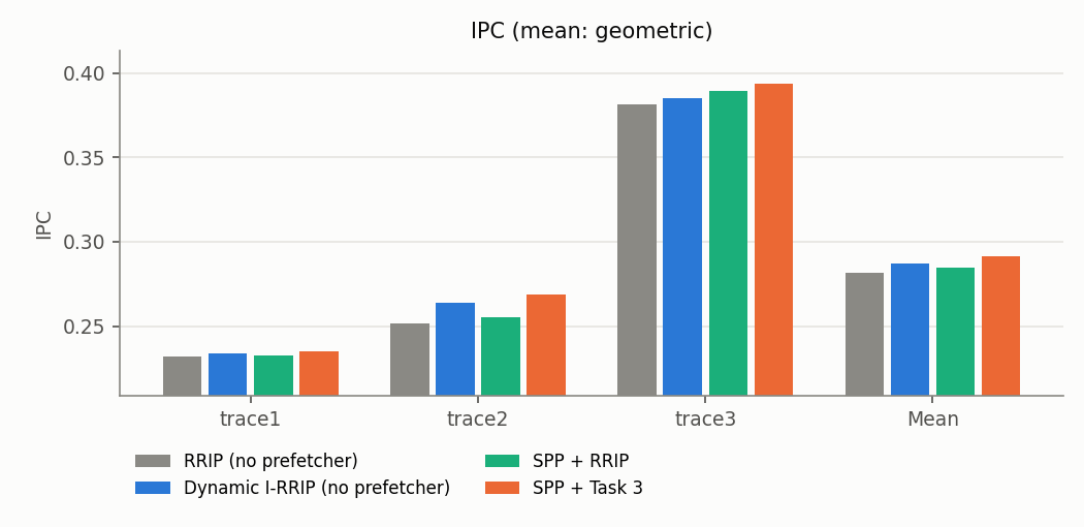
Table 2: Comprehensive Performance and Cache Hierarchy Evaluation for Task 3.

2. Visualizations and Metric Analysis

“‘latex

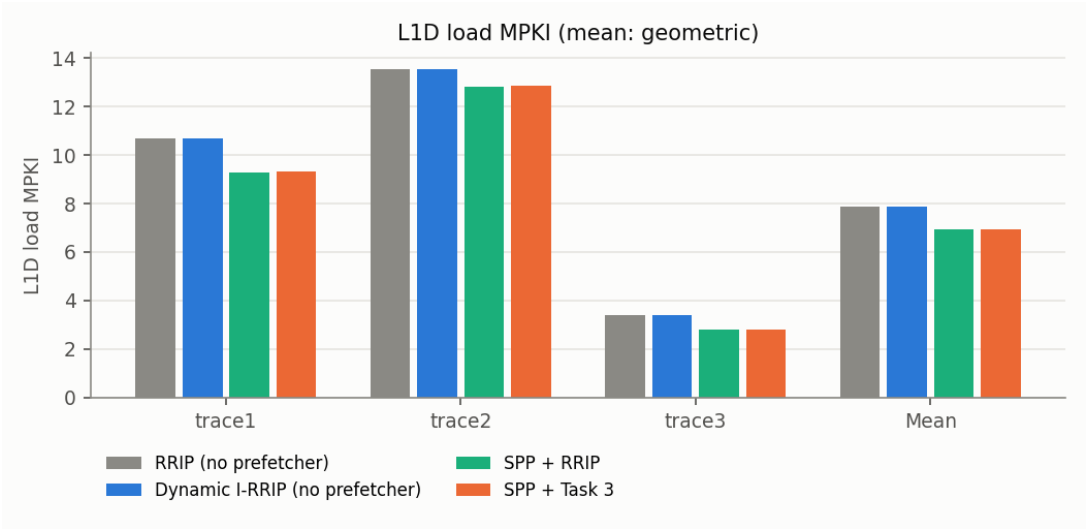


(a) Speedup over RRIP Baseline (Geometric Mean)

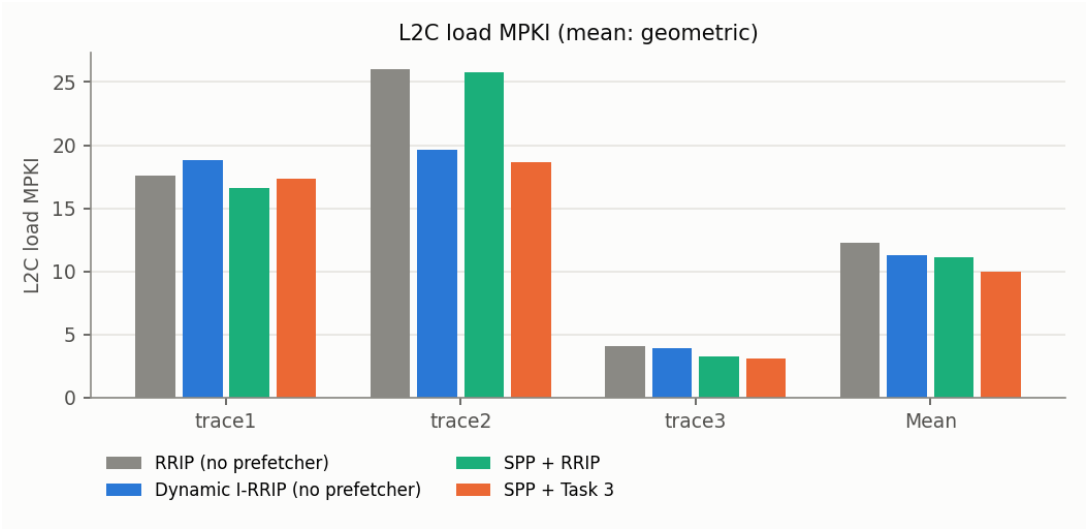


(b) IPC Across Traces (Geometric Mean)

Figure 3: Performance Metrics of the Integrated SPP + Task 3 Replacement Policy.

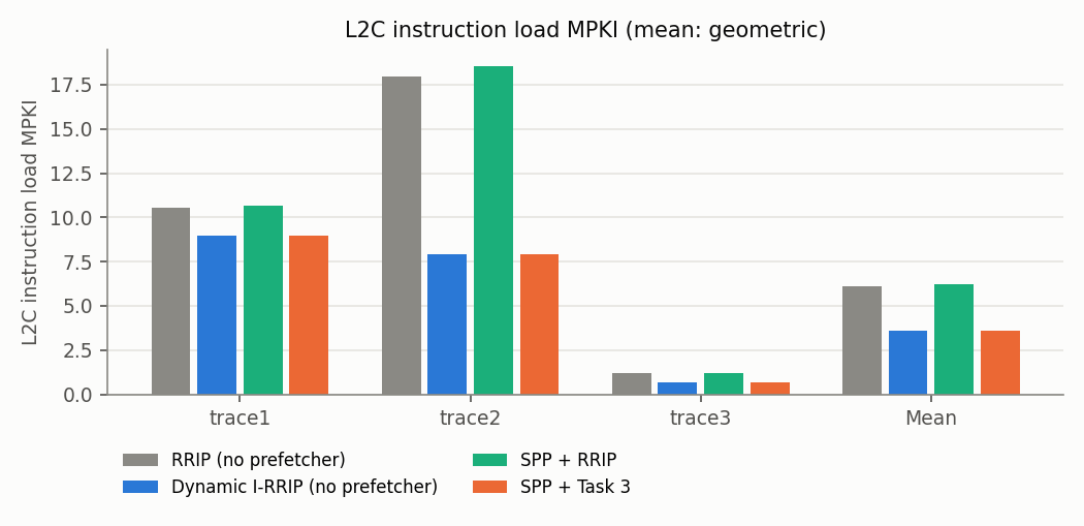


(a) L1D Load MPKI (Lower is Better)

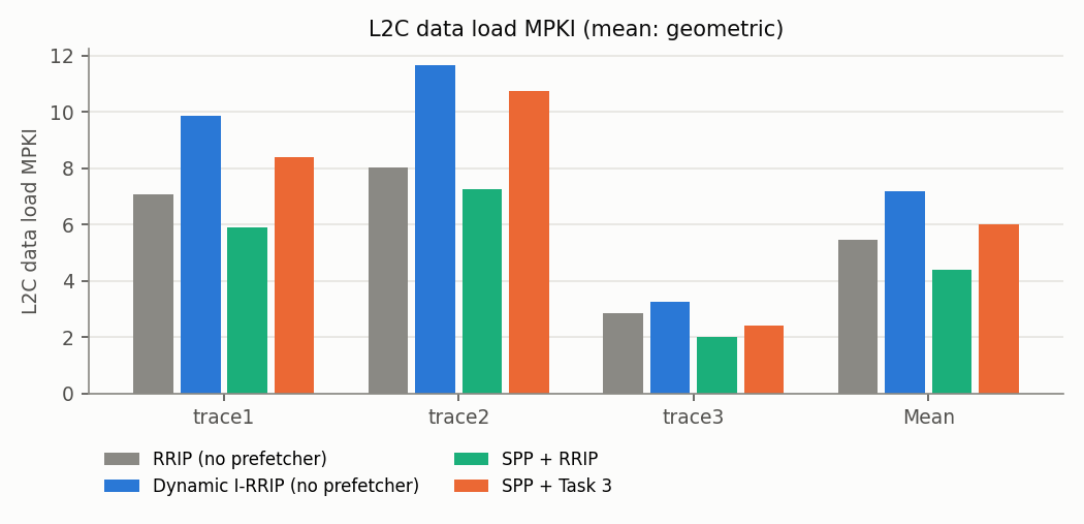


(b) Total L2C Load MPKI (Lower is Better)

Figure 4: L1D and L2C Cache Miss Metrics.

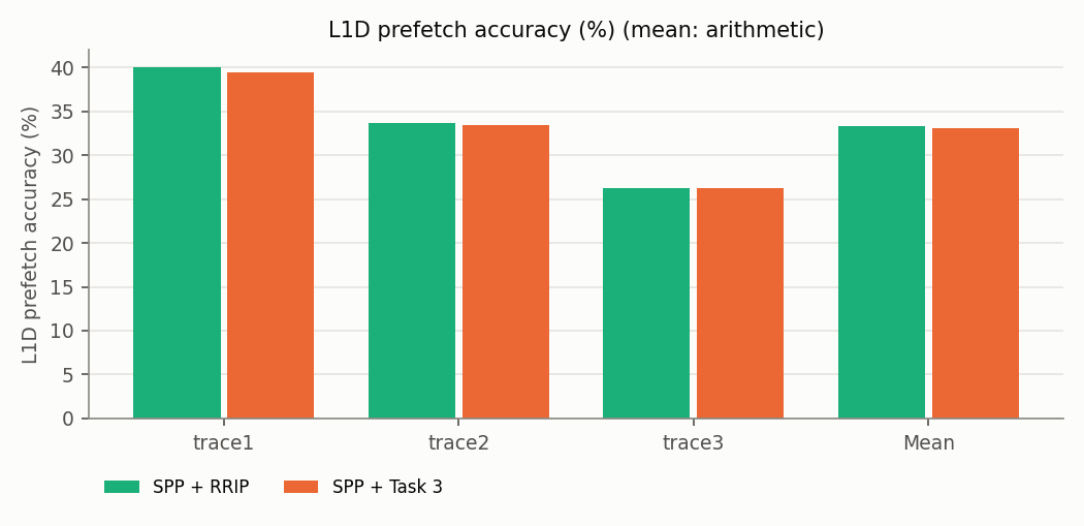


(a) L2C Instruction Load MPKI

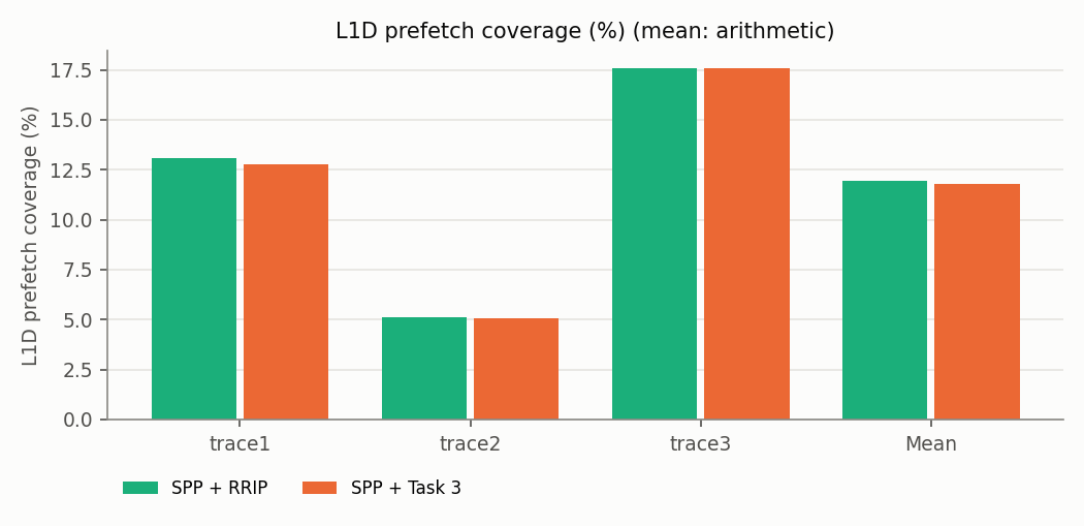


(b) L2C Data Load MPKI

Figure 5: Instruction and Data Load Misses in the L2 Cache.



(a) L1D Prefetch Accuracy (%)



(b) L1D Prefetch Coverage (%)

Figure 6: SPP L1D Prefetch Quality under Standalone RRIP vs. Prefetch-Aware Task 3.

‘‘‘

3. Microarchitectural Analysis

The primary objective of Task 3 is to design an integrated mechanism where the L2 replacement policy cooperates with the L1D prefetcher, strictly satisfying the functional constraint:

$$\text{IPC}_{\text{SPP+Task3}} > \max\left(\text{IPC}_{\text{Dyn-IRRIP}}, \text{IPC}_{\text{SPP+RRIP}}\right)$$

As demonstrated by the geometric mean in Table 2, **SPP + Task 3 achieves an IPC of 0.2917 (1.0361× speedup)** outperforming both standalone Dynamic I-RRIP (0.2873, 1.0205×) and standalone SPP (0.2847, 1.0114×) across every individual benchmark trace:

- **Trace 1 (Instruction-Constrained Regime):** Standalone SPP yields almost no performance benefit (1.0008×) because execution stalls primarily on L2 instruction misses (10.56 MPKI). Dynamic I-RRIP reduces instruction MPKI to 8.97, raising speedup to 1.0061×. Task 3 combines these improvements: it retains the full L1D miss reduction of

SPP (cutting L1D MPKI from 10.68 to 9.32) while simultaneously driving L2C instruction MPKI down to 8.95, achieving a peak speedup of 1.0116×.

- **Trace 2 (Massive Instruction Footprint):** Trace 2 suffers from heavy frontend starvation, exhibiting a baseline L2C instruction MPKI of 17.98. While standalone SPP provides only a 1.0130× speedup (as it only prefetches data), Task 3 reduces L2 instruction MPKI by over 56% (down to 7.91) while sustaining an L1D prefetch coverage of 5.06%. This combination yields an exceptional 1.0664× **speedup (0.2686 IPC)**, our largest single-trace improvement.

- **Trace 3 (Prefetch-Dominated Regime):** In Trace 3, the instruction footprint is small (baseline L2 instruction MPKI is only 1.19), making SPP data prefetching the dominant factor (17.56% coverage, 1.0203× speedup). Task 3 maintains complete prefetch fidelity (17.57% coverage, 26.19% accuracy) while cutting residual L2 instruction misses by 45% (0.6566 MPKI). The overall L2C MPKI drops to an all-time low of 3.06, producing a 1.0310× **speedup**.

4. Rationale Behind Prefetch Packet Handling in the Replacement Policy

Running a data prefetcher with an instruction-conscious replacement policy creates two distinct failures:

1. *Instruction Crowding:* If prefetched data lines are inserted with elevated priority (`RRPV` ≤ 2), speculative data competes directly with instruction lines. When cache pressure peaks, instruction lines are evicted to preserve speculative data, spiking instruction MPKI (+13% in our early exploration) and causing severe IPC degradation. Because SPP issues thousands of speculative requests, counting prefetch packets in the leader sets artificially inflates total data accesses, which decreases the instruction intensity metric and tricks Dynamic I-RRIP into switching off.

To eliminate these bottlenecks, our final Task 3 implementation incorporates following mechanisms:

- **Speculative Insertion at `maxRRPV` (RRPV = 3):** All incoming prefetch fills (`type == PREFETCH`) are inserted strictly at `maxRRPV` (3). This guarantees that speculative lines never share the initial priority tier of demand instruction lines (`RRPV = 2`). If an SPP prefetch is inaccurate, it resides at the eviction frontier and is discarded without displacing useful cache content. If it is accurate, a subsequent demand `LOAD` hits it and promotes it to `RRPV = 0`.

- **Victim Search:** Inserting prefetches at `RRPV = 3` exposes them to early eviction under standard I-RRIP, which evicts data lines at `RRPV = 3` first. To give prefetches time to be referenced without harming instructions, we track prefetched blocks using a 1-bit metadata array (`is_prefetch[set][way]`). During victim selection among lines at `maxRRPV`, the replacement policy evaluates candidates in three passes:

  1. **Pass 1:** Evict stale demand data lines (`!instruction && !is_prefetch`).
  2. **Pass 2:** Evict prefetch data lines (`!instruction && is_prefetch`).
  3. **Pass 3:** Evict instruction lines (`instruction`).

  This ordering mathematically ensures that instructions receive strict priority over all data lines, while newly prefetched data receives some priority over older dead demand data.

- **Hit Promotion:** When an incoming prefetch packet hits an already-cached line (`hit && type == PREFETCH`), we execute an early return without updating the RRPV. A prefetch hit merely confirms that the prefetcher predicted a block already resident in the cache; promoting it to `RRPV = 0` without real demand would artificially extend the lifetime of stagnant blocks, causing cache pollution.

By combining above things we successfully merged SPP and Dynamic I-RRIP, yielding the maximum IPC across all benchmark traces.